PRACTITIONER GUIDE & REFERENCE IMPLEMENTATION

Building a CTI Program: An Engineering Blueprint

Stitching together the threat intelligence platform, SIEM, EDR, cloud runtime sensors and custom host telemetry, and the teams that act on intelligence, with working code for every integration.

Author	Pat Campbell
Repository	cti-engineering-blueprint (lifecycle toolkit + integration layer + sensor content)
Version	1.0 · November 2026

All hosts, users, indicators and inventories in examples are synthetic (RFC 5737 IP ranges, .example / .test domains).

Contents

1 How to Use This Guide 4

2 Program Foundations 5

2.1 Mission and requirements 5
2.2 Roles 5
2.3 The operating loop 5

3 Reference Architecture 7

3.1 Components 7
3.2 Data flows 7
3.3 Design principles 8

4 Implementation Roadmap 9

5 Connectors: Configure or Build 10

5.1 Decision framework 10
5.2 Connector catalog 10
5.3 Integration engineering standards 11

6 Data Economics: Index, Pivot or Drop 12

6.1 The problem 12
6.2 Three tiers for every field 12
6.3 The field budget method 12

7 The Pivot Layer 14

7.1 Why pivot 14
7.2 The five questions 14
7.3 Vendor implementation details 14
7.4 Entry points 15
7.5 Write-back 16

8 Fidelity Engineering 17

8.1 The admin problem 17
8.2 Composition over single signals 17
8.3 Three alerts, one name 17
8.4 Relationship to risk-based alerting 18

9 Coverage Engineering 19

9.1 Every product assumes telemetry 19
9.2 Reconciliation 19
9.3 Compensating telemetry by platform 20
9.4 Wiz Defend and the Wiz Runtime Sensor 21
9.5 Sysmon as a second data point 21
9.6 Linux without EDR 22

10 TIP Engineering 24

10.1 The TIP lifecycle 24
10.2 TIP-to-control rules that prevent outages 24

10.3 Feedback into the TIP ... 25

11 Working with the Other Teams ... 26

11.1 Security engineering ... 26
11.2 Detection engineering ... 26
11.3 Threat hunting ... 26
11.4 Red team and purple team ... 26
11.5 SOC and incident response ... 27
11.6 Where stringing processes together pays off ... 27

12 Operating the Platform ... 28

13 Engineering Through an Attacker's Eyes ... 29

14 Field Notes ... 30

15 Repository Reference ... 32

15.1 Commands ... 32
15.2 Module map ... 32
15.3 Verification performed ... 33
15.4 Known limits ... 33

16 Sources ... 34

1 How to Use This Guide

This guide is a practitioner's blueprint for building a cyber threat intelligence (CTI) program from the engineering side. It covers how to stitch together the threat intelligence platform (TIP), the SIEM, EDR, cloud runtime sensors, custom host telemetry and the teams that consume intelligence, so that external data turns into decisions people can trust.

It assumes a large, regulated enterprise: tens of thousands of endpoints, several EDR products from acquisitions and platform choices, a SIEM priced by volume, cloud workloads, legacy Linux, appliances, a mainframe and OT. Every recommendation is generic and vendor-neutral where it can be. Vendors are named where you need the specifics to build (API endpoints, SDK methods, scopes), because a guide that only says "integrate your EDR" doesn't help anyone build anything.

What's in the repository

Area	Path	What it does
Lifecycle toolkit	`cti.py`, `ctikit/`, `hypotheses/`, `detections/`	Hypothesis to collection, scoring, STIX 2.1, Sigma to SPL, YARA, reports
Integration layer	`fabric_cli.py`, `fabric/`	EDR pivot adapters (CrowdStrike, SentinelOne, Carbon Black, Defender), Wiz runtime detections, TIP lookups, fidelity scoring, write-back, field budgeting, coverage reconciliation, connector health
Sensor content	`sensors/`	Modular Sysmon config builder, Sysmon for Linux config, auditd rules, osquery pack, Splunk inputs and TIP-match search
SIEM app	`splunk_app/TA-cti-edrpivot`	`| edrpivot` custom search command
Tests	`tests/`	29 tests: unit, end to end, fake SDK objects, mock vendor APIs
Docs	`docs/`	This guide, the practitioner runbook

Everything runs offline against synthetic data (RFC 5737 IP addresses, `.example` and `.test` domains, invented hosts and users). Live mode reads credentials from environment variables.

Reading the tips. Short boxes marked **Tip** are the condensed version of a section: one or two sentences you can say out loud to a technical peer. They're collected again in the Field Notes chapter.

> **Tip:** CTI engineering is the work of making intelligence executable: normalized, scored, delivered to the tool that acts on it, and measured. Analysis decides what matters; engineering makes sure it lands.

2 Program Foundations

2.1 Mission and requirements

A CTI program exists to answer the questions decision-makers have about adversaries, and to turn those answers into controls. Engineering serves that. Before building anything, write down:

- **Priority Intelligence Requirements (PIRs):** the standing questions, owned by named stakeholders. For example: "Which actors target our sector through identity channels, and would we detect them?"
- **Consumers and their formats:** the SOC needs context at triage time, detection engineering needs rules and test data, IR needs scoping queries, leadership needs a BLUF and a risk statement.
- **Decisions the data drives:** block, detect, hunt, patch, change a process, or accept risk.

Requirements decide which feeds are worth paying for, which fields are worth indexing and which integrations matter first.

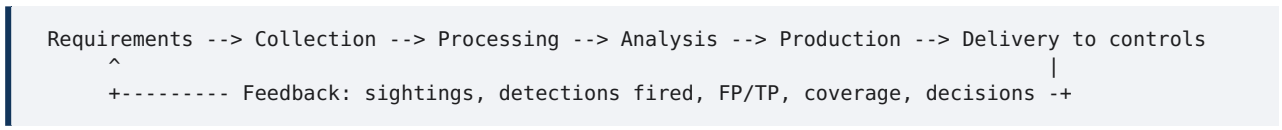
2.2 Roles

Role	Owns
CTI analysts	Requirements, research, analytic judgments, finished intelligence
CTI engineer / TIP owner	Platform configuration, data model, connectors, automation, scoring and lifecycle rules, pipeline metrics
Detection engineering	Detection content in production, tuning, the detection repo
Threat hunting	Hypothesis-driven hunts, baselines, new detections from hunts
SOC	Triage and first response, the main consumer of enrichment
Incident response	Investigation and containment, the main producer of new internal intel
Red / purple team	Emulation of real procedures, validation of detection coverage
Security engineering	EDR, SIEM, sensor and logging platforms, agent deployment

The CTI engineer sits at the junction. They translate analyst needs into platform capabilities, and translate platform limits back into analyst expectations.

> **Tip:** The TIP owner is a translator. Analysts ask for "a better feed"; the engineering answer is usually better normalization, scoring and delivery of the feeds you already have.

2.3 The operating loop



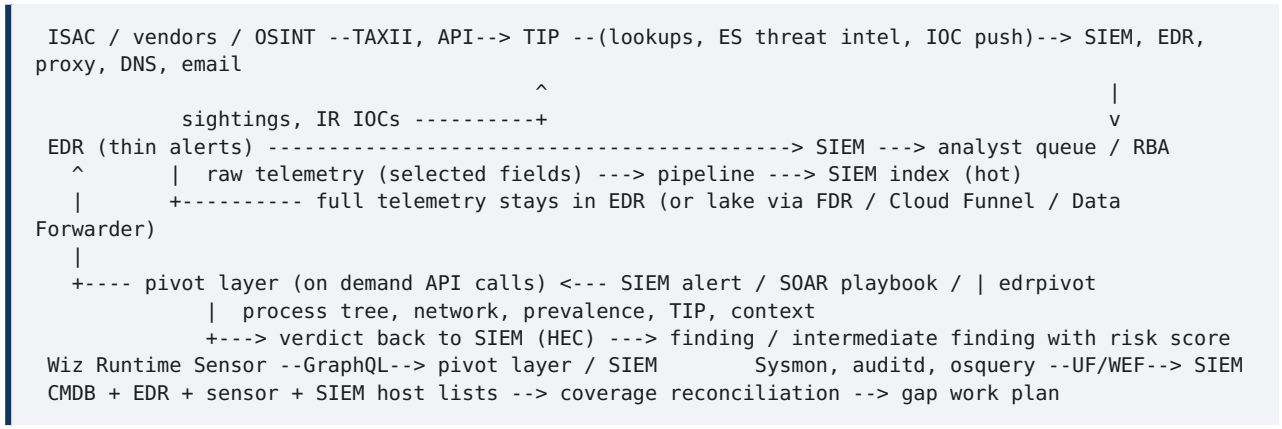
Engineering owns the arrows: how data moves, how it's transformed, and how feedback flows back. Most programs fail on the right-hand side. Intel gets produced but never reaches a control in a usable form, and nobody measures whether it did.

3 Reference Architecture

3.1 Components

Layer	Examples	Role in the program
External intel	ISAC TAXII feeds, commercial intel, open sources (abuse.ch, CISA KEV), VirusTotal / Google Threat Intelligence	Raw and finished intelligence
TIP	Anomali ThreatStream, ThreatConnect, OpenCTI, MISP	System of record: normalize, score, relate, age, share
SIEM	Splunk Enterprise Security, Microsoft Sentinel, Google SecOps	Correlation, detection, analyst queue
EDR / XDR	CrowdStrike Falcon, SentinelOne Singularity, Carbon Black Cloud, Microsoft Defender for Endpoint	Deep endpoint telemetry, response actions
Cloud runtime	Wiz Defend with the Wiz Runtime Sensor (eBPF), CNAPP posture data	Workloads where traditional EDR is weak or absent
Custom host telemetry	Sysmon (Windows and Linux), auditd, osquery, syslog	Coverage for hosts without EDR, a second data point where EDR exists
SOAR	Splunk SOAR, Cortex XSOAR, Tines	Playbooks: enrichment, routing, response with approvals
Data pipeline / lake	Cribl, Splunk Ingest Actions or Edge Processor, S3 or a data lake, Federated Search	Route, reduce and tier data by value
Context	CMDB, IAM / directory, change management, vulnerability management	Who, what, how critical, and whether it was expected
Integration layer	`fabric/` in this repo	Pivots, fidelity scoring, coverage, connector health

3.2 Data flows



3.3 Design principles

1. **Normalize once.** Every source maps into one data model, so integrations grow as sources plus consumers, not sources times consumers.
2. **The TIP is the system of record for intelligence.** Custom code reads and writes it. It doesn't become a shadow TIP.
3. **Index what you detect on; pivot for the rest.** Pay to index the fields that detections and routing need. Fetch everything else from the EDR API when an alert asks for it.
4. **Fidelity comes from combination.** No single weak signal is worth an analyst's time; signals plus context plus rarity plus intel are.
5. **Coverage is a first-class metric.** Every product assumes telemetry exists. Prove it does, and track where it doesn't.
6. **Read-only by default; response is gated.** Containment needs an approval reference.
7. **Everything observable, everything owned.** Each connector has a health check, a service-level objective, an owner and a runbook.

> **Tip:** I design the integration layer so the analytics are vendor-blind. Adapters absorb each EDR's quirks, and everything above them sees one data model. That's what lets you survive an EDR migration or a merger.

4 Implementation Roadmap

A phased plan. Each phase ends with something measurable in production.

Phase	Weeks	Deliverables	Exit metric
0. Discover	1-4	Inventory every feed, connector, integration and owner. Pull the PIR list. Map data flows as they are, not as the diagram says. Baseline SIEM ingest by source and field.	Integration register complete, with an owner for every row
1. Foundation	4-10	Data model and scoring rules agreed with the SOC and DE. TIP-to-SIEM lookup with context fields. Connector health checks. Secrets moved to a vault. CI for detection content.	Health dashboard live; zero connectors without an owner
2. Pivot layer	8-16	EDR adapters for each platform. `| edrpivot` command and a SOAR playbook. Verdict write-back to a dedicated index. A first finding-based detection on pivot verdicts.	Median triage time on pilot alert types reduced; verdicts on 100% of pilot alerts
3. Coverage	10-20	CMDB, EDR, sensor and SIEM reconciliation, run weekly. Sysmon (native or standalone) where EDR is blocked. Wiz Runtime Sensor on Kubernetes and cloud Linux. auditd and osquery on legacy Linux.	Coverage percentage by criticality tier reported monthly; critical BLIND count trending down
4. Economics	12-24	Field budget per EDR source. Pipeline allowlists. Hot / warm / lake tiers. Federated search for cold data.	GB per day reduced without losing any field a detection uses
5. Feedback	Ongoing	Sightings back to the TIP. FP/TP per intel source. Coverage and detection trends. Purple team validation cadence.	Program metrics reviewed quarterly with stakeholders

> **Tip:** First 30 days I don't build anything. I inventory every integration, who owns it, and when it last failed. Most "we need a new connector" problems are really "the connector we have broke six months ago and nobody noticed."

5 Connectors: Configure or Build

5.1 Decision framework

Use the vendor connector when...	Build custom when...
The job is high-volume, standard IOC matching (TIP to SIEM, TIP to EDR blocklists)	You need your own context in the payload: hypothesis, score factors, expiry, action
The vendor maintains it and supports upgrades	The logic is yours: scoring, relevance, lifecycle, fidelity
The data shape is standard (STIX/TAXII, CEF, the vendor's Splunk add-on)	Two tools have no supported path between them
Operations staff need to support it without a developer	You need on-demand depth (pivots) rather than bulk streaming

Both usually coexist. The vendor connector moves the volume, and custom code adds the judgment.

> **Tip:** I don't rewrite what the vendor maintains. I write code where the vendor doesn't know my environment: my scoring, my context, my lifecycle and my detection-as-code.

5.2 Connector catalog

Path	Typical mechanism	Notes
ISAC / vendor feeds to TIP	TAXII 2.1 collections, vendor APIs, TIP-native feed connectors	TAXII is poll-based; persist `added_after` for incremental pulls
TIP to SIEM	Vendor app (for example the ThreatStream integration for Splunk), lookup CSV or KV store, Splunk ES threat intelligence collections	Carry score, tags, actor, expiry and TLP with every indicator
TIP to EDR	Native integration or the EDR custom-IOC API	EDRs cap custom IOC counts. Push only live, high-confidence, action-appropriate indicators, with expiry
EDR alerts to SIEM	Vendor add-on or streaming API	Keep it thin: IDs, host, user, hash, tactic
EDR raw telemetry to lake / SIEM	CrowdStrike Falcon Data Replicator, SentinelOne Cloud Funnel or SDL, Carbon Black Cloud Data Forwarder, Defender streaming API (Event Hubs or storage)	Route through a pipeline with a field allowlist
EDR deep data on demand	Vendor API / SDK (this repo's `fabric/edr`)	The pivot layer
Wiz to SIEM / pivot layer	Wiz GraphQL API (OAuth client credentials), vendor integrations	Sensor detections plus cloud inventory
Custom host telemetry to SIEM	Splunk Universal Forwarder, Windows Event Forwarding, syslog	Sysmon, auditd, osquery

Path	Typical mechanism	Notes
SIEM to TIP (sightings)	API write-back, SOAR	Feeds scoring and actor tracking
Pivot verdicts to SIEM	HTTP Event Collector into a dedicated index	ES detections turn verdicts into findings

5.3 Integration engineering standards

- **Service accounts with least privilege.** Read-only scopes for collectors and pivots. A separate, approval-gated identity for response actions. Document every scope.
- **Secrets in a vault**, injected at runtime and rotated, with expiry alerts. Never in `.conf` files or code. In Splunk apps, use the storage/passwords endpoint.
- **Rate-limit budgets below vendor limits**, so a busy alert day doesn't lock analysts out of the console. Microsoft Graph advanced hunting, for example, allows about 45 calls a minute and 10 minutes of query time an hour; the Defender adapter budgets 30 a minute.
- **Retry with backoff and `Retry-After`.** Treat 429 and 5xx as transient and 4xx as configuration errors.
- **Idempotency.** Use deterministic IDs and upserts, so re-runs update rather than duplicate.
- **Parameterized queries.** Alert fields are attacker-influenced. Pass them as query parameters (CQL `?param` arguments) or escape them properly (KQL and PowerQuery literals). Never concatenate raw strings into queries.
- **Audit logging** of every outbound call: who, what, why. Never the secret.
- **Dry run by default** for anything that writes to a TIP, SIEM or control.
- **Health checks** with an SLO per connector and alerts on lag, not just failure.
- **Tests** against fake SDK objects and mock servers, in CI.

> **Tip:** Alert fields are attacker-controlled input. A hostname or command line goes into an EDR query as a bound parameter, never string-concatenated. I treat my own SOC tooling as an injection surface.

6 Data Economics: Index, Pivot or Drop

6.1 The problem

EDR telemetry is enormous. A single process-creation event from a modern EDR carries 30 to 60 fields, and an enterprise produces billions of events a day. SIEM licensing is tied to ingest volume or compute, so forwarding everything is rarely affordable. Forwarding too little blinds detections. The way through is to treat each field as a cost line and justify it.

6.2 Three tiers for every field

Tier	Where it lives	Use it for
INDEX (hot)	SIEM index	Fields detections reference, and routing or pivot keys (device ID, process key, host, user, hash, timestamp, event type)
PIVOT (on demand)	Stays in the EDR (or the vendor's data lake)	Everything else useful for investigation: full ancestry, module loads, extra hashes, network detail. Fetched by API when an alert asks
LAKE (cold)	Object storage via FDR, Cloud Funnel, Data Forwarder or a streaming API, searchable with federated search	Long-retention hunting, compliance, retro-hunts against new intel
DROP	Nowhere	Near-empty or pure sensor-housekeeping fields

6.3 The field budget method

`fabric/field_budget.py` turns this into a repeatable decision:

1. Sample raw EDR events as they would be forwarded.
2. Measure each field: fill rate, average bytes per event, distinct values.
3. Parse the detections you actually run (SPL, Sigma, KQL) and record which fields they reference.
4. Classify each field as INDEX, PIVOT or DROP.
5. Project GB per day at your event volume, for everything versus INDEX only.

Demo output on synthetic CrowdStrike-style process events at 2 billion events a day:

Measure	Value
Fields in sample	33
INDEX fields	11 (CommandLine, SHA256HashData, ImageFileName, event_simpleName, ParentBaseFileName, timestamp, TargetProcessId, ComputerName, IntegrityLevel, UserName, aid)
PIVOT fields	22 (e.g. SHA1HashData, MD5HashData, cid, GrandParentBaseFileName, ProcessStartTime, ConfigBuild)
GB/day, all fields	2,037.62

Measure	Value
GB/day, INDEX only	914.35
Reduction	55.1% with no detection losing a field

The INDEX allowlist becomes the field filter in your pipeline: Splunk Ingest Actions or Edge Processor, Cribl, or the EDR's export filter. Re-run it whenever detection content changes. A new detection that needs a PIVOT field promotes it to INDEX through a reviewed change, not an ad hoc forwarder edit.

> **Tip:** I classify every forwarded field as index, pivot or drop, based on whether a deployed detection uses it. On EDR data that's typically a large volume cut with no detection losing a field. Whatever investigation needs beyond that, I fetch from the EDR API on demand.

> **Tip:** Cost reduction is a detection-engineering change, not a logging change. The allowlist is generated from the detection repo, so the two can't drift apart.

7 The Pivot Layer

7.1 Why pivot

An EDR alert reaches the SIEM as a thin record: alert ID, host, user, maybe a hash and a process key. That's enough to route, but not enough to decide. The full context (who spawned the process, what it spawned, where it connected, how rare the binary is) is already stored in the EDR and already paid for. The pivot layer fetches it on demand, through the vendor's API or SDK, at the moment an alert or analyst needs it.

This is also how you analyze data that is available through the API but never extracted into alert fields.

> **Tip:** The SIEM gets the thin alert; the EDR API gets the questions. I use the vendor's Python client to pull the process tree, network activity and hash prevalence on demand, so I'm not paying to index fields that only matter for one alert in ten thousand.

7.2 The five questions

Every adapter in `fabric/edr/` implements the same interface:

Method	Question	Why it matters
`host_context`	What is this machine?	OS, group, tags, last seen; joins to the CMDB
`process_tree`	Who launched this, and what did it launch?	Office to script host to LOLBin chains are the classic tell
`network_for`	Where did these processes connect?	IPs and domains to check against the TIP
`hash_prevalence`	How many hosts have run this binary?	A binary on one host out of 50,000 is interesting; one on all of them isn't
`contain`	Isolate the host	Gated: refuses without `allow_response` and an approval reference

7.3 Vendor implementation details

	CrowdStrike Falcon	SentinelOne Singularity	Carbon Black Cloud
Python client	FalconPy (`crowdstrike-falconpy`), official SDK with token handling	No widely used official SDK; REST with stdlib	Carbon Black Cloud Python
Auth	OAuth2 API client (client ID and secret), scoped	Management API: `ApiToken` . SDL: Bearer Log Read key	API key: `SECRET/ID` plus o
Alert detail	`Alerts.get_alerts_v2(composite_ids=...)`	`GET /web/api/v2.1/threats?ids=`	`api.select(Alert, id)`

	CrowdStrike Falcon	SentinelOne Singularity	Carbon Black Cloud
Telemetry query	NG-SIEM CQL jobs: `NGSIEM.start_search` then poll `get_search_status` (repo `investigate_view` for endpoint events)	SDL PowerQuery `POST /api/powerQuery`	Process search: `api.select(Process).wh`
Best pivot key	`aid` + `TargetProcessId`	Storyline ID (`src.process.storyline.id`): the whole story in one query	`process_guid`, globally u
Tree method	Walk `ParentProcessId` upward; query children by `ParentProcessId`	One storyline query returns every process creation	SDK `.parents`, `.childre`
Network	`NetworkConnectIP4`, `DnsRequest` by `ContextProcessId`	`IP Connect`, `DNS Resolved` events in the storyline	`process.events(event_`
Containment	`Hosts.perform_action(action_name="contain")`	`POST agents/actions/disconnect`	`Device.quarantine(Tru`
Gotchas	Query jobs are async; stop jobs on timeout. Pass CQL `?params`	Legacy Deep Visibility APIs deprecated Feb 15, 2026 and stop working Feb 15, 2027; migrate to Long Running Query, Saved Searches or SDL	Several fields come back as

> **Tip:** Each EDR has a better pivot key than the OS PID, which recycles. Falcon has aid plus TargetProcessId, Carbon Black has process_guid, SentinelOne has the storyline ID. With SentinelOne one storyline query returns the whole attack chain.

> **Tip:** If you still have scripts on SentinelOne's legacy Deep Visibility API, that API stops working in February 2027. Inventory anything calling `/dv/` now and migrate it to the long-running query API or SDL.

7.4 Entry points

The same `pivot.run()` backs three entry points:

1. **Splunk custom search command** (`| edrpivot`, in `splunk_app/TA-cti-edrpivot`): analysts and scheduled searches call it inline. It caps rows per search (`maxrows`) to protect API quotas and never breaks the search on error.
2. **SOAR playbook step:** enrich every new alert of selected types before a human opens it.
3. **Pivot service:** a scheduled job that pivots new thin alerts and writes verdicts to the SIEM.

Each pivot has a time budget (default 90 seconds). Slow steps are skipped when it runs out, and skipped steps are recorded in the verdict. Partial evidence with an honest list of gaps beats a timeout.

7.5 Write-back

Verdicts go to a dedicated `cti_pivot` index through HTTP Event Collector, with `risk_object`, `risk_object_type` and `risk_score` fields. A detection in the SIEM reads that index and creates the analyst-facing finding. The SIEM keeps ownership of risk thresholds and queueing; the pivot service stays a replaceable data producer.

8 Fidelity Engineering

8.1 The admin problem

"PowerShell with an encoded command" is what attackers do. It's also what management tools and administrators with poor security habits do. Alert on it alone and the queue floods; suppress it and you miss intrusions. Some attacks are only detectable from all the data together, because any single piece looks like an admin.

8.2 Composition over single signals

`fabric/fidelity.py` composes evidence from the pivot:

- **Behavior signals** from the process tree and command lines: Office spawning a script host, encoded PowerShell, LOLBin proxy execution from user-writable paths, download cradles, discovery commands, payload staging, and credential dumping (including `comsvcs.dll MiniDump` against LSASS).
- **Rarity:** an unsigned binary seen on two hosts or fewer.
- **Intelligence:** TIP matches on any IP, domain or hash in the tree, weighted by TIP score.
- **Context:** asset criticality, privileged identity, admin host or PAW, approved change window, unknown asset.

Rules

Rule	Why
Hard signals (high-confidence TIP match, credential dumping) are never discounted	An admin running an LSASS dump is still an incident
Soft signals are discounted by explaining context: privileged user on an admin host (x0.5), inside an approved change (x0.7)	Expected admin work stops reaching the queue
A privileged account used off an admin host adds +15	That's a posture problem at best and credential exposure at worst
Crown-jewel assets amplify (x1.3; high criticality x1.15)	Same behavior, higher stakes
An asset missing from the CMDB adds +10	Unknown assets are a finding in themselves
Escalate on score of 70 or more across two or more tactics, or any hard signal; review at 40-69; suppress below 40 as a recorded risk event	Tactic diversity is a strong discriminator; nothing is silently dropped

8.3 Three alerts, one name

The demo sends three thin alerts, all named "Suspicious PowerShell Execution", from three different EDRs. At alert time they're indistinguishable. After the pivot:

Alert	Host / user	Decision	Score	Signals	Context modifiers	Reason
ALERT-ATTACK-001	FIN-WS-0231 / jdoe	**escalate**	100	office_spawns_script_host(40), encoded_powershell(20), lolbin_proxy_execution(25), rare_unsigned_binary(20), tip_match(50,HARD)	high-criticality asset: x1.15	hard signal present - context cannot discount it
ALERT-ADMIN-002	ADM-JMP-02 / adm-rsmith	**suppress**	11	encoded_powershell(20), discovery_commands(10)	privileged user on admin host: soft signals x0.5; inside approved change CHG0049812: soft signals x0.7; high-criticality asset: x1.15	explained by context / below threshold - kept as risk record, not queued
ALERT-GREY-003	OPS-WS-0110 / adm-tlee	**review**	60	download_cradle(20), payload_staged_to_writable_path(15), discovery_commands(10)	privileged account used off an admin host: +15	material signal, context incomplete - human review

> **Tip:** Same alert name, three outcomes. The attack escalates on a TIP-matched C2 plus an Office to PowerShell to rundll32 chain. The admin on a PAW inside an approved change is suppressed with the change number as the reason. The privileged account pulling a script from the internet on an ordinary workstation goes to a human. The decision is in the context, not the signature.

> **Tip:** Context can discount soft signals but never hard ones. That one rule stops "it was an admin" from becoming the attacker's best camouflage.

8.4 Relationship to risk-based alerting

This is the same idea as risk-based alerting in Splunk Enterprise Security. In ES 8.x terms, detections produce **intermediate findings** (risk events that never hit the analyst queue on their own), and **finding-based detections** raise a **finding** when an entity's combined risk, or its count of distinct ATT&CK tactics, crosses a threshold. ES 8 replaced the older "notable event" and "risk event" terms with findings and intermediate findings. The pivot layer adds evidence the SIEM never indexed. Writing verdicts back with risk fields lets ES aggregate them with everything else it knows about the entity.

> **Tip:** I feed pivot verdicts into risk-based alerting rather than straight to the queue. Low-score verdicts become intermediate findings on the entity, so three weak days in a row still add up to a finding.

9 Coverage Engineering

9.1 Every product assumes telemetry

Intelligence, detections and hunts all assume the data exists. In a large estate it often doesn't. Linux kernels the EDR doesn't support, appliances that forbid agents, legacy servers frozen by change policy, Kubernetes nodes that churn hourly, mainframes, OT, and assets nobody knew existed are the places attackers go once they've learned where the sensors are.

9.2 Reconciliation

`fabric/coverage.py` joins four inventories (CMDB, EDR consoles, runtime sensors, SIEM host-by-sourcetype counts) and classifies every asset:

Class	Meaning
EDR_HEALTHY	Agent present and checked in recently
EDR_STALE	Agent present but silent beyond the threshold (broken, offline, decommissioned?)
SENSOR_ONLY	No EDR, but a runtime sensor such as Wiz covers it
LOGS_ONLY	No agent; host logs (Sysmon, auditd, osquery, syslog) reach the SIEM
BLIND	Nothing
UNMANAGED	Seen by a tool but missing from the CMDB

Demo result on a synthetic 67-asset estate:

Class	Count
EDR_HEALTHY	54
EDR_STALE	2
SENSOR_ONLY	4
LOGS_ONLY	4
BLIND	3
UNMANAGED	1

Healthy EDR coverage: **80.6%**. Any telemetry: **92.5%**.

Gap asset	Platform	Criticality	Class	Recommended control
dev-laptop-x9	unknown	0	UNMANAGED	Identify owner and platform first - an unknown asset is itself the finding
f5-lb-03	appliance	4	BLIND	Vendor syslog + config backup diffing + NetFlow/NDR around it; agents usually unsupported

Gap asset	Platform	Criticality	Class	Recommended control
lnx-db-ora02	linux	5	BLIND	EDR Linux sensor if the kernel is supported; else an eBPF runtime sensor (e.g. Wiz Runtime Sensor) or Sysmon for Linux on modern kernels; on old kernels without usable eBPF (e.g. RHEL 6) use auditd + osquery to the SIEM
lnx-k8s-node-00	container	4	SENSOR_ONLY	Runtime sensor as a Kubernetes DaemonSet (e.g. Wiz Runtime Sensor) + k8s audit logs
lnx-k8s-node-01	container	4	SENSOR_ONLY	Runtime sensor as a Kubernetes DaemonSet (e.g. Wiz Runtime Sensor) + k8s audit logs
lnx-k8s-node-02	container	4	SENSOR_ONLY	Runtime sensor as a Kubernetes DaemonSet (e.g. Wiz Runtime Sensor) + k8s audit logs
lnx-k8s-node-03	container	4	SENSOR_ONLY	Runtime sensor as a Kubernetes DaemonSet (e.g. Wiz Runtime Sensor) + k8s audit logs
lnx-rhel6-batch01	linux	5	LOGS_ONLY	EDR Linux sensor if the kernel is supported; else an eBPF runtime sensor (e.g. Wiz Runtime Sensor) or Sysmon for Linux on modern kernels; on old kernels without usable eBPF (e.g. RHEL 6) use auditd + osquery to the SIEM
srv-win-legacy01	windows	4	LOGS_ONLY	Deploy EDR; if blocked, native/standalone Sysmon with the repo config + Windows Event Forwarding
swift-gw-01	appliance	5	LOGS_ONLY	Vendor syslog + config backup diffing + NetFlow/NDR around it; agents usually unsupported
ws-0040	windows	3	EDR_STALE	Investigate agent health / decommission status
ws-0041	windows	3	EDR_STALE	Investigate agent health / decommission status
zos-prod-a	mainframe	5	LOGS_ONLY	SMF / RACF (or ACF2/Top Secret) events forwarded via syslog connector

Report coverage by criticality tier, not just as one percentage. Ninety-five percent coverage that misses the payment gateway is a failing grade.

> **Tip:** I reconcile CMDB, EDR, sensor and SIEM host lists every week. The output isn't a percentage; it's a work list of blind critical assets, each with an owner and a compensating control.

> **Tip:** Stale agents count as gaps. An agent that hasn't checked in for ten days is a dashboard green light covering a blind spot.

9.3 Compensating telemetry by platform

Platform	Preferred	When the agent is not possible
Windows	EDR	Sysmon (native optional feature on Windows 11 since 2026, or standalone) with a modular config, shipped with the Splunk UF or Windows Event Forwarding
Modern Linux	EDR Linux sensor	eBPF runtime sensor (Wiz Runtime Sensor), or Sysmon for Linux (eBPF), plus osquery

Platform	Preferred	When the agent is not possible
Legacy Linux (old kernels without usable eBPF, for example RHEL 6)	n/a	auditd with ATT&CK-keyed rules, plus osquery, to the SIEM
Kubernetes / containers	Runtime sensor as a DaemonSet	Kubernetes audit logs, cloud control-plane logs
Appliances (load balancers, gateways)	Vendor logging	Syslog, configuration-backup diffing, NetFlow / NDR around the device
Mainframe	n/a	SMF and RACF (or ACF2 / Top Secret) events via a syslog connector
OT	n/a	Passive network monitoring at the segment boundary. Never active agents without OT engineering approval

9.4 Wiz Defend and the Wiz Runtime Sensor

Wiz started as agentless cloud posture and inventory (useful in its own right as an inventory source for reconciliation). Wiz Defend adds detection and response, and the Wiz Runtime Sensor is a lightweight eBPF sensor that runs inside workloads: Linux VMs, containers and Kubernetes (deployed as a DaemonSet), with a Windows sensor added in June 2026. It sees process execution, system calls and network activity at runtime. Wiz Forensics, generally available since May 2026, captures a forensic package at detection time: the triggering binary or script, the process tree, shell history and system logs.

Integration (`fabric/wiz.py`):

- **Auth:** OAuth2 client credentials at `https://auth.app.wiz.io/oauth/token` with `audience=wiz-api`, using a service account with read scopes (`read:detections`, `read:cloud_events_sensor`, `read:issues`, `read:resources`).
- **API:** GraphQL at `https://api.<region>.app.wiz.io/graphql` with cursor pagination. The schema is unversioned and changes, so the repo keeps the detections query in `queries/wiz_detections.graphql` for you to replace with the version exported from the Wiz API explorer, instead of hard-coding it.
- **Normalization:** `detection_to_alert()` maps Wiz detections into the same `ThinAlert` model, so cloud workload detections flow through the same fidelity scoring and write-back as endpoint alerts.

> **Tip:** Where the EDR can't go (ephemeral containers, Kubernetes nodes, some Linux), I use an eBPF runtime sensor like Wiz's and normalize its detections into the same pipeline. One fidelity model across endpoints and cloud workloads.

9.5 Sysmon as a second data point

Sysmon earns its place in two roles. It's primary telemetry where EDR is blocked, and an independent second source where EDR exists. That's useful when an EDR is tampered with, when you need a field the EDR doesn't expose, or when you need to validate one sensor against another.

Native Sysmon. Microsoft announced in late 2025 that Sysmon would become a native Windows feature, and in 2026 Windows 11 gained it as an optional feature updated through Windows Update. It uses the same engine, the same XML configuration and the same `Microsoft-Windows-Sysmon/Operational` channel. The standalone Sysinternals build remains supported, which matters for servers and older systems. Check native availability for your server OS versions before planning on it.

Modular configuration (`sensors/sysmon/`). One config file per event type or use case, assembled by `build_sysmon.py`, which also checks the XML and reports ATT&CK coverage from the rule names:

Module	Event IDs	Intent
`01_process_create`	1	Exclude-mode: log all process creation except known noise
`03_network_connect`	3	Include-mode: script hosts, LOLBins and RMM tools that make network connections
`08_10_injection_and_lsass`	8, 10	Remote threads; LSASS access with memory-read rights
`11_file_create`	11	Startup folders, scheduled tasks, scripts and executables staged to disk
`12_14_registry_persistence`	12-14	Run keys, services, COM hijack
`22_dns`	22	DNS queries minus noise; the cheapest TIP domain match on hosts without EDR
`25_tampering`	25	Process hollowing and herpaderping

Rule `name` attributes carry `technique_id=...,technique_name=...`, so every event's `RuleName` field maps directly to ATT&CK in the SIEM. This follows the convention popularized by community configurations. The build reports 13 tagged techniques for the shipped modules.

> **Tip:** I build Sysmon configs the way I build detections: modular, reviewed in merge requests, built and validated in CI, with ATT&CK IDs in the rule names so every event arrives pre-mapped.

9.6 Linux without EDR

- **Sysmon for Linux** (`sensors/linux/sysmon-linux-config.xml`) uses eBPF and the same XML filtering model. Events go to syslog. Watch shells, curl, wget, Python, netcat and socat making network connections, and persistence file writes (cron, `authorized_keys`, systemd units, `ld.so.preload`).
- **auditd** (`sensors/linux/audit.rules`) works on old kernels. It covers execve for users and root, identity file changes, sudoers, SSH keys, cron, systemd, preload hijack and kernel module loads. Each rule's key carries an ATT&CK ID, and the config is set immutable (`-e 2`) to resist tampering. Test execve volume on a representative host first.
- **osquery** (`sensors/linux/osquery-cti-pack.conf`) runs scheduled state queries: listening ports, authorized keys, crontab, SUID binaries, external sockets, and binaries deleted after execution. Differential results go to the SIEM.

- **TIP matching for these hosts** (`sensors/splunk/tip_match_custom_sensors.spl`) joins Sysmon DNS and network events and osquery sockets to the TIP lookup, so hosts without EDR still get IOC coverage.

> **Tip:** On a box that can't take an EDR, I still get three things: execution (auditd or Sysmon for Linux), persistence changes, and outbound connections matched against the TIP. That's enough to catch most intrusions and to know where to look.

10 TIP Engineering

10.1 The TIP lifecycle

A TIP's job, stated as engineering capabilities: **ingestion, normalization, enrichment, scoring, prioritization, validation, publication, expiration, retention, tasking and historical context.** Each one is a design decision:

Capability	Decision to make and write down
Ingestion	Which feeds, how often, incremental watermarks, failure alerting
Normalization	One data model; type detection; refang; drop non-routable and documentation ranges
Enrichment	Which enrichers (VT/GTI, passive DNS, WHOIS age, ASN, internal sightings), caching, quota
Scoring	Explainable factors and weights, agreed with the SOC
Prioritization	Relevance to requirements and sector
Validation	Analyst approval for new automation; import-with-approval workflows
Publication	Which controls get which actions: block, detect or hunt
Expiration	TTL by indicator type, following the Pyramid of Pain
Retention	Expired indicators stay searchable for retro-hunts and attribution
Tasking	RFIs and hypotheses as tracked objects with owners and due dates
Historical context	Actor, campaign and incident relationships; sightings over time

The lifecycle toolkit in this repo (`ctikit/`) implements most of this: collectors for abuse.ch, CISA KEV, ThreatStream and TAXII 2.1; dedupe with corroboration; VT/GTI enrichment; a five-factor score with action rules and TTL; STIX 2.1 export with deterministic IDs; Sigma to SPL; YARA; and a Splunk lookup that the pivot layer's `CsvTIP` reads directly. That last link closes the loop: research produces scored intel, and the pivot layer uses it at triage time.

10.2 TIP-to-control rules that prevent outages

- Never push private, loopback or documentation ranges, or your own address space.
- Never hash-block signed dual-use software (RMM tools, admin utilities). Make those behavioral detections and hunt context instead.
- IP blocks need a higher bar than domain blocks: shared hosting and cloud IPs carry the most collateral risk.
- Every pushed indicator carries an expiry. EDR custom-IOC slots are finite.
- New automation starts in approval mode; move a flow to direct import only after it has a track record.

> **Tip:** Score and action are different fields. A signed ScreenConnect installer can be 100% malicious in context and still must never be hash-blocked. It becomes a behavioral detection.

10.3 Feedback into the TIP

- **Sightings:** every TIP match in the SIEM or pivot layer is written back to the TIP as a sighting, which raises the indicator's local relevance and dates the activity.
- **False positives:** analyst dispositions on CTI-sourced findings update source reliability.
- **IR output:** IOCs and TTPs from closed incidents become private intel linked to the incident.

11 Working with the Other Teams

11.1 Security engineering

Security engineering owns the EDR, SIEM and sensor platforms. CTI engineering depends on them. Agree on:

- **API access:** service accounts, scopes, rate budgets, and who rotates secrets.
- **Field allowlists:** the field budget output becomes their pipeline configuration, through change control.
- **Agent coverage:** the weekly coverage report goes to them as a work queue; blind critical assets get tickets.
- **Sensor content:** Sysmon, auditd and osquery configs live in Git with CI. Security engineering deploys and CTI proposes.

11.2 Detection engineering

- CTI hands over procedures, not technique names: Sigma merge requests with replay data and the hypothesis ID.
- Detection engineering owns promotion; generated searches ship disabled.
- Pivot verdicts become an input to finding-based detections that DE owns.
- The ATT&CK coverage map is generated from rule tags, so it can't drift from what's deployed.

11.3 Threat hunting

- CTI supplies hypotheses and hunt packages (hypothesis, data sources, window, queries, exit criteria).
- The pivot layer and the Splunk command give hunters API depth without indexing cost.
- Every successful hunt ends as a detection or a documented baseline.

11.4 Red team and purple team

- CTI gives the red team current, procedure-level tradecraft from the actors on the PIR list.
- Purple team exercises execute those procedures against production telemetry and check three outcomes for each: detected, logged but not alerted, or not logged. Each outcome updates the coverage map and creates engineering tickets.
- Test the fidelity model too. Run the attack and the admin look-alike side by side and confirm one escalates and the other doesn't.
- Test the sensors themselves: tamper with Sysmon or the EDR agent, and confirm the coverage report and health checks notice.

> **Tip:** In purple team I score each procedure three ways: detected, logged but not alerted, or not logged. "Logged but not alerted" is detection-engineering work. "Not logged" is a coverage or field-budget problem. Different owners, different fixes.

11.5 SOC and incident response

- The SOC sees verdicts, reasons and evidence in the finding, not a raw IOC hit. Every suppress decision records why.
- IR gets pivot evidence packages and the approval-gated containment path.
- Both send dispositions back, which calibrate scoring and fidelity weights.

11.6 Where stringing processes together pays off

Seam	Manual today	Automated
Feed to TIP	Analysts import CSVs	Collectors with normalize, dedupe and score on ingest
TIP to controls	Manual blocklist updates	Lifecycle-aware publishing with expiry
Alert to evidence	Analyst opens three consoles	Pivot on arrival; evidence in the finding
Evidence to decision	Gut feel	Explainable fidelity model plus human review band
Incident to intel	Copy-paste into the TIP	IR IOCs and TTPs written back with links
Research to detection	Email with a description	Sigma MR plus replay data, CI-validated
Detection to coverage	Spreadsheet	Generated ATT&CK layer from rule tags
Inventory to coverage	Annual audit	Weekly reconciliation with tickets

Keep humans at the judgment points: publishing to blocking controls, containment, attribution, and anything that goes to leadership.

12 Operating the Platform

- **Health:** every connector has an authenticated ping, a "newest data seen" timestamp and an SLO (`fabric/health.py`). Alert on lag, not just failure.
- **Ownership:** an integration register with owner, scopes, secret expiry, SLO and runbook link for every connector.
- **Change control:** detection, sensor and integration changes go through merge requests and CI. Production writes need a ticket reference.
- **Testing:** fake SDK objects and mock servers in CI (this repo's tests cover CrowdStrike, Carbon Black, SentinelOne, Defender, Wiz and ThreatStream paths), plus replay data for detections.
- **Audit:** every outbound API call is logged with purpose. Response actions are logged with their approval reference.

Metrics

Type	Metric
Pipeline	Feed health, dedupe ratio, corroboration rate, time from ingest to control
Pivot	Pivots per day, median pivot time, step error rate, API quota headroom
Fidelity	Escalate / review / suppress split; TP rate by decision; suppressed-then-confirmed count (must trend to zero)
Coverage	Healthy EDR %, any-telemetry %, BLIND count by criticality tier, stale agents
Economics	GB/day by source, INDEX vs PIVOT field share, cost per detection
Program	PIRs answered, detections shipped from CTI, decisions changed

> **Tip:** The metric I watch most closely on a fidelity model is "suppressed, later confirmed malicious." It must be zero or trending to zero, and every instance gets a post-mortem on the weights.

13 Engineering Through an Attacker's Eyes

Engineering choices in a CTI platform are security decisions. An adversary who understands your pipeline will aim for its seams. Design against that:

- **Attackers go where the sensors aren't.** Coverage gaps are attack surface. Blind Linux boxes, appliances and unmanaged assets are where long dwell times come from.
- **Attackers blend in with admins.** LOLBins, RMM tools, PowerShell remoting and valid accounts are chosen because they look like operations. Context discounts must never apply to hard signals.
- **Attackers tamper with telemetry.** Stopping Sysmon, unloading drivers and silencing agents. Stale-agent detection, Sysmon service and config-change alerts, auditd immutability and a second, independent data source exist for this.
- **Attackers poison inputs.** Alert fields, command lines and threat reports are untrusted. Parameterize queries, and treat reports fed to LLM-based extraction as prompt-injection-capable content.
- **Attackers abuse your automation.** If containment is automatic, an attacker can make you isolate your own servers. Response stays approval-gated.
- **Attackers exploit stale intel.** Expired IOCs pushed to blocking controls burn analyst trust until the SOC ignores CTI. Lifecycle discipline is credibility.

> **Tip:** Coming from offensive work, the first question I ask about any pipeline is "how would I hide from this?" Usually the answer is a host without a sensor, an admin tool, or an alert rule that context can talk down. That's where the engineering goes.

14 Field Notes

The short versions. Each is a sentence or two you can say to a technical peer.

Program

- Intelligence answers a stakeholder's question and changes a decision. A feed is data.
- The TIP owner translates. Analysts ask for a better feed; the answer is usually better processing of the feeds you have.
- First month: inventory every integration, its owner and its last failure, before building anything.

Architecture and connectors

- Normalize once. Integrations scale as sources plus consumers, not sources times consumers.
- Vendor connectors move volume. Custom code adds my context, scoring, lifecycle and detection-as-code.
- Adapters absorb vendor quirks so the analytics are vendor-blind. That's how you survive an EDR migration.
- Least-privilege service accounts, secrets in a vault, rate budgets below vendor limits, dry run by default.
- Alert fields are attacker-controlled. Bound parameters in every EDR query.

Data economics

- Every forwarded field is index, pivot or drop, decided by whether a deployed detection uses it.
- The field allowlist is generated from the detection repo, so cost cuts can't silently break detections.
- Raw telemetry can go to a lake through FDR, Cloud Funnel or Data Forwarder for long-tail hunting; the SIEM stays hot and lean.

Pivot layer

- The SIEM gets the thin alert, and the EDR API gets the questions: process tree, network, prevalence.
- Use the vendor's best pivot key: Falcon aid plus process ID, Carbon Black process_guid, SentinelOne storyline.
- One pivot implementation behind three entry points: Splunk command, SOAR step, scheduled service.
- Time-box pivots. Partial evidence with a gap list beats a timeout.
- SentinelOne legacy Deep Visibility APIs stop working February 2027. Migrate now.

Fidelity

- No single weak signal is worth an analyst. Signals plus context plus rarity plus intel are.
- Context discounts soft signals, never hard ones. An admin dumping LSASS is still an incident.

- Same alert name, three outcomes: escalate, suppress with a change number, or send to a human.
- Verdicts feed risk-based alerting as intermediate findings, so weak signals accumulate on an entity.
- Watch "suppressed, later confirmed" like a hawk.

Coverage

- Reconcile CMDB, EDR, sensors and SIEM weekly. The output is a work list, not a percentage.
- Stale agents are gaps.
- Where EDR can't go, use eBPF runtime sensors like Wiz, Sysmon for Linux, or auditd plus osquery on old kernels.
- Sysmon is a second, independent data point even where EDR exists, and it's now a native optional feature in Windows 11.
- Modular Sysmon config in Git, built in CI, with ATT&CK IDs in rule names.
- Hosts without EDR still get TIP coverage through DNS, network and socket telemetry joined to the TIP lookup.

Teams

- Hand detection engineering procedures, replay data and a Sigma MR, not technique names.
- Purple team outcomes are detected, logged but not alerted, or not logged. Each has a different owner.
- Keep humans at publish-to-block, containment, attribution and leadership communication.

15 Repository Reference

15.1 Commands

```
pip install -r requirements.txt

# Lifecycle toolkit
python cti.py run hypotheses/HYP-2026-001-helpdesk-idp-takeover.yaml --offline
python cti.py validate

# Integration layer
python fabric_cli.py pivot --mock
python fabric_cli.py pivot --alert alert.json --edr crowdstrike --tip threatstream --context ./
context
python fabric_cli.py field-budget --sample events.jsonl --detections detections/ --events-per-day
2000000000
python fabric_cli.py coverage --inventory ./inventory
python fabric_cli.py wiz-pull --since 2026-11-05T00:00:00Z

# Sensors
python sensors/sysmon/build_sysmon.py --schema 4.90

# Tests (29)
python -m unittest discover -s tests -v
```

15.2 Module map

Module	Purpose
`fabric/common.py`	Data model, stdlib HTTP with retry, token-bucket rate limits, audit log, response gating
`fabric/edr/base.py`	Adapter interface, mock adapter, factory
`fabric/edr/crowdstrike.py`	FalconPy: Alerts, Hosts, NG-SIEM CQL jobs with `?params`
`fabric/edr/sentinelone.py`	Management API + SDL PowerQuery, storyline pivots
`fabric/edr/carbonblack.py`	CBC SDK: Alert, Device, Process (parents, children, events)
`fabric/edr/defender.py`	Graph runHuntingQuery with client-credentials token, KQL escaping
`fabric/wiz.py`	Wiz OAuth + GraphQL pagination, detection normalization
`fabric/tip.py`	ThreatStream lookups with cache; CSV lookup from ctikit
`fabric/context.py`	Asset, identity and change-window context
`fabric/fidelity.py`	Signals, hard/soft rules, context modifiers, decision
`fabric/pivot.py`	Orchestration with time budget and partial-failure tolerance
`fabric/writeback.py`	HEC events with risk fields
`fabric/field_budget.py`	INDEX / PIVOT / DROP classification and GB/day projection

Module	Purpose
`fabric/coverage.py`	Inventory reconciliation and compensating-control plan
`fabric/health.py`	Connector health with SLOs
`splunk_app/TA-cti-edrpivot`	`| edrpivot` custom streaming command
`sensors/`	Sysmon modules and builder, Sysmon for Linux, auditd, osquery, Splunk inputs, TIP match search

15.3 Verification performed

- Full test suite: **29 tests, all passing**, run on the packaged repo.
- CrowdStrike and Carbon Black adapters tested with fake SDK objects matching method names verified against FalconPy 1.6.6 and cbc_sdk 1.5.10.
- SentinelOne, Defender (Graph) and Wiz (OAuth + GraphQL pagination) adapters tested against a local mock server returning documented response shapes.
- Fidelity model: attack, admin and grey-area scenarios produce escalate, suppress and review; a hard signal escalates despite admin context.
- Pivot tolerates a failing EDR step and records it; containment refuses without allow_response and an approval reference.
- Sysmon config built from modules and parsed as XML; Sysmon for Linux XML and osquery JSON validated; Splunk command compiles.
- Lifecycle toolkit: STIX bundle validated with stix2, Sigma converted to SPL with pySigma, YARA compiled with YARA-X.

15.4 Known limits

- Vendor field names (SDL schema, CQL event fields, CBC event fields, Wiz GraphQL schema) change. Each adapter documents the names it uses; verify them in your tenant before production.
- The Wiz detections query is a template; export the current one from the Wiz API explorer.
- The Sysmon `schemaversion` must match what `sysmon -s` reports on your build.
- Fidelity weights are starting points. Calibrate them quarterly against dispositions.
- Mock data is synthetic and illustrative.

16 Sources

Facts about current products and APIs were checked on November 5, 2026.

- CrowdStrike FalconPy service classes (Alerts, Hosts, NGSIEM) and the StartSearchV1 / GetSearchStatusV1 operations: github.com/CrowdStrike/falconpy, developer.crowdstrike.com; NG-SIEM repositories (Ansible ngsiem_search module docs). Method names verified against FalconPy 1.6.6.
- SentinelOne deprecation of legacy Deep Visibility and S1QL 1.0 (deprecated Feb 15, 2026; stops Feb 15, 2027): N-able status notice summarizing SentinelOne release notes. SDL PowerQuery `/api/powerQuery`: Tenzir and integrator documentation.
- Carbon Black Cloud Python SDK 1.5.10 (March 2026) searching guide and changelog: carbon-black-cloud-python-sdk.readthedocs.io. Process methods verified against the installed SDK.
- Microsoft Graph runHuntingQuery and security API limits: learn.microsoft.com/graph/api/security-security-runhuntingquery and the security API overview.
- Splunk Enterprise Security 8 findings, intermediate findings and finding-based detections: help.splunk.com (ES 8.x administer docs); ES 7.x risk-based alerting thresholds.
- Wiz Runtime Sensor, Wiz Defend, Windows sensor and Forensics GA: wiz.io blog "Is Wiz a Runtime Security Tool?" (2026). Wiz OAuth and GraphQL endpoints and scopes: RunReveal, XSOAR, D3 and Grafana integration docs.
- Native Sysmon in Windows: Mark Russinovich's 2025 announcement (via TechSpot, HackMag) and 2026 coverage of the Windows 11 optional feature.
- Anomali ThreatStream API: docs.anomali.com.